

A PROJECT REPORT ON

# **SMART HOUSEHOLD ENERGY FORECASTING AND ANOMALY DETECTION**

Domain: Energy / IoT + ML

Submitted in partial fulfilment of the requirements for the award of the degree of

**B.Sc. / B.E. / MCA – COMPUTER SCIENCE**

Submitted by

**STUDENT NAME**

(Register No: XXXXXXXXX)

Under the guidance of

**GUIDE NAME, M.Sc., M.Phil.,**

Assistant Professor

**DEPARTMENT OF COMPUTER SCIENCE**

**COLLEGE NAME**

2025 – 2026

# CONTENTS

| CHAPTER    | TITLE                                     | PAGE      |
|------------|-------------------------------------------|-----------|
| <b>I</b>   | <b>INTRODUCTION</b>                       | <b>1</b>  |
|            | 1.1 History of Domain                     |           |
|            | 1.2 Overview of the Project               |           |
|            | 1.3 Motivation                            |           |
|            | 1.4 Aim of the Project                    |           |
|            | 1.5 Objectives                            |           |
|            | 1.6 Literature Review                     |           |
| <b>II</b>  | <b>SOFTWARE PROJECT PLAN</b>              | <b>5</b>  |
|            | 2.1 Existing System                       |           |
|            | 2.2 Proposed System                       |           |
|            | 2.3 Project Schedule                      |           |
| <b>III</b> | <b>CUSTOMER REQUIREMENT DETERMINATION</b> | <b>7</b>  |
|            | 3.1 Functional Requirements               |           |
|            | 3.2 Non-functional Requirements           |           |
|            | 3.3 User Requirements                     |           |
|            | 3.4 System Requirements                   |           |
| <b>IV</b>  | <b>SOFTWARE REQUIREMENT SPECIFICATION</b> | <b>9</b>  |
|            | 4.1 Hardware Specification                |           |
|            | 4.2 IoT Hardware Components               |           |
|            | 4.3 Software Specification                |           |
| <b>V</b>   | <b>SYSTEM ANALYSIS</b>                    | <b>10</b> |

|             |                                          |           |
|-------------|------------------------------------------|-----------|
|             | 5.1 Feasibility Study                    |           |
|             | 5.2 About the Software                   |           |
| <b>VI</b>   | <b>SYSTEM DESIGN</b>                     | <b>14</b> |
|             | 6.1 UML Diagram                          |           |
|             | 6.2 System Architecture                  |           |
|             | 6.3 Data Flow Diagram                    |           |
|             | 6.4 ER Diagram                           |           |
|             | 6.5 Table Design                         |           |
|             | 6.6 Algorithm Design                     |           |
|             | 6.7 Modules                              |           |
| <b>VII</b>  | <b>IMPLEMENTATION AND TESTING</b>        | <b>23</b> |
|             | 7.1 Introduction                         |           |
|             | 7.2 Implementation                       |           |
|             | 7.3 Test Cases                           |           |
|             | 7.4 Source Code                          |           |
|             | 7.5 Screen Layout                        |           |
| <b>VIII</b> | <b>CONCLUSION AND FUTURE ENHANCEMENT</b> | <b>30</b> |
|             | 8.1 Conclusion                           |           |
|             | 8.2 Future Enhancement                   |           |
| <b>IX</b>   | <b>BIBLIOGRAPHY</b>                      | <b>31</b> |

# **CHAPTER I**

## **INTRODUCTION**

Electricity consumption in Indian homes has grown rapidly with the increasing use of air conditioners, refrigerators, water heaters, washing machines, induction stoves and electric vehicles. Most households receive only a bi-monthly or monthly bill showing total units consumed, with no insight into which appliances consume the most, when the consumption happens, or whether anything is wrong.

Faulty appliances such as a refrigerator with a failing compressor or a worn door seal, a water heater with a defective thermostat, or a motor pump that runs dry can waste a lot of power unnoticed for months. Electricity tariffs are slab-based - the rate per unit increases as consumption crosses certain limits - so a small increase in usage can raise the bill significantly. With time-of-day tariffs being introduced, peak-hour usage will also cost more.

This project, WattWise, uses smart plugs or a smart meter built with an ESP32 and PZEM-004T / ACS712 sensors to record household consumption. The system forecasts next-day and month-end usage using Prophet and LSTM models, detects abnormal consumption using statistical thresholds and Isolation Forest, estimates the bill using tariff slabs, and suggests load shifting and appliance tips to cut the electricity bill.

### **LIMITATIONS**

1. Users see only a monthly bill with no insight.
2. Faulty appliances waste power unnoticed.
3. Peak-hour and slab-crossing usage raises cost.
4. No prediction of the bill before it arrives.

### **1.1 History of Domain**

Electricity metering began with electromechanical induction meters in the late nineteenth century and remained largely unchanged for a century. Electronic meters and

Automatic Meter Reading (AMR) came in the 1990s, followed by Advanced Metering Infrastructure (AMI) smart meters with two-way communication in the 2000s.

India launched the Smart Meter National Programme to replace conventional meters with prepaid smart meters, and the Electricity (Rights of Consumers) Amendment Rules, 2023 introduced time-of-day tariffs. Research on Non-Intrusive Load Monitoring (NILM, Hart, 1992), load forecasting and anomaly detection has shown how fine-grained consumption data can help households save energy.

## 1.2 Overview of the Project

A whole-home node (ESP32 + PZEM-004T on the main supply) measures voltage, current, power, energy and power factor, and optional smart plug nodes (ESP32 + ACS712 + relay) measure individual appliances such as the refrigerator, AC and water heater. Readings are published every 10 seconds via MQTT. The server aggregates them into minute, hourly and daily values per device.

A Prophet model forecasts next-day hourly consumption and the month-end total using daily and weekly seasonality and temperature as an external regressor; an LSTM model is compared as an alternative. Anomaly detection uses statistical thresholds (e.g., consumption above mean + 3 standard deviations for that hour) and an Isolation Forest on appliance features such as duty cycle and average on-power (for example, a refrigerator compressor running longer than usual). The bill is estimated using the TNEB domestic tariff slabs, and recommendations are generated, such as shifting washing machine and pump usage to off-peak hours or checking the refrigerator door seal.

## 1.3 Motivation

Households can save a meaningful part of their electricity bill if they know where and when energy is used and are warned about waste.

- **Motivation:** To turn raw consumption data into forecasts, alerts and simple advice that reduce electricity cost.

- **Outcome:** Lower bills, early detection of faulty appliances and reduced peak demand on the grid.

## 1.4 Aim of the Project

The project is entitled as “**SMART HOUSEHOLD ENERGY FORECASTING AND ANOMALY DETECTION**”. The aim of this project is to develop an IoT-based household energy monitoring system that forecasts future consumption, detects abnormal usage and faulty appliances, estimates the electricity bill according to tariff slabs, and recommends actions to reduce cost.

## 1.5 Objectives

- To measure whole-home and appliance-level power using PZEM-004T and ACS712 with ESP32.
- To aggregate readings by hour and device.
- To forecast next-day and month-end usage with Prophet/LSTM.
- To detect anomalies using thresholds and Isolation Forest.
- To estimate bills using tariff slabs and recommend savings.

## 1.6 Literature Review

**Hart (1992):** Introduced Non-Intrusive Appliance Load Monitoring, which disaggregates total household load into individual appliances by detecting step changes in power.

**Kelly and Knottenbelt (2015):** Released the UK-DALE dataset with whole-house and individual appliance power demand from five UK homes, used widely for energy research.

**Kong et al. (2019):** Applied LSTM recurrent neural networks to short-term residential load forecasting and showed improved accuracy over conventional methods for individual households.

**Taylor and Letham (2018):** Proposed Prophet for forecasting at scale with interpretable trend and seasonality components and support for external regressors.

**Liu, Ting and Zhou (2008):** Proposed Isolation Forest for anomaly detection, efficient for detecting unusual patterns in high-dimensional data.

**Darby (2006):** Reviewed the effectiveness of feedback on energy consumption and found that direct, real-time feedback typically leads to savings of a few percent to over ten percent.

## **CHAPTER II**

### **SOFTWARE PROJECT PLAN**

#### **2.1 Existing System**

In the existing system, electricity board staff read the meter once in two months and the consumer pays according to the total units. Consumers cannot see consumption by appliance or time, cannot predict the bill, and do not know if an appliance is faulty until it breaks down or the bill jumps.

#### **DISADVANTAGES**

- No real-time or appliance-wise information.
- Bill surprises due to slab crossing.
- Faulty appliances waste energy unnoticed.
- No guidance for reducing consumption.
- No data for peak-hour management.

#### **2.2 Proposed System**

The proposed WattWise system monitors consumption in real time, forecasts usage and bills, detects anomalies and faulty appliances, and provides personalised recommendations through a web dashboard and alerts.

#### **ADVANTAGES**

- Real-time and appliance-level visibility.
- Bill prediction before the slab is crossed.
- Early warning of faulty appliances.
- Personalised saving tips and load shifting.
- Low-cost DIY hardware.

## 2.3 Project Schedule

The development of WattWise follows the iterative waterfall model. Each phase produces a deliverable that is reviewed before the next phase begins, and feedback from testing is fed back into design and implementation. The planned schedule is shown below.

| S.No | Phase                                       | Duration     |
|------|---------------------------------------------|--------------|
| 1    | Requirement analysis and problem study      | Week 1 - 2   |
| 2    | Literature survey and existing system study | Week 3 - 4   |
| 3    | Dataset collection / hardware procurement   | Week 5 - 6   |
| 4    | System design (DFD, UML, ER, table design)  | Week 7 - 8   |
| 5    | Implementation of modules                   | Week 9 - 12  |
| 6    | Testing and model evaluation                | Week 13 - 14 |
| 7    | Documentation and final review              | Week 15 - 16 |

## CHAPTER III

### CUSTOMER REQUIREMENT DETERMINATION

#### 3.1 Functional Requirements

Functional requirements define the specific features and operations a system must perform to meet business and user needs. They describe what the system should do and how it interacts with users or other systems. They focus on the behaviour and functionality of the system and represent features that can be directly observed and tested in the final product. The functional requirements of the WattWise system are listed below.

- **Power measurement:** Measure voltage, current, power and energy.
- **Data upload:** Send readings to the cloud every 10 seconds.
- **Aggregation:** Aggregate by hour, day and device.
- **Forecasting:** Forecast next-day and month-end usage.
- **Anomaly detection:** Detect abnormal consumption and appliance faults.
- **Bill estimation:** Estimate bill using tariff slabs.
- **Recommendations:** Suggest load shifting and appliance tips.
- **Dashboard:** Daily and monthly charts and alerts.

#### 3.2 Non-functional Requirements

Non-functional requirements determine the performance standards and quality attributes of software, e.g. system usability, effectiveness, security, scalability, etc. While functional requirements determine what the system does, non-functional requirements describe HOW the system does it. If an application does not meet non-functional requirements, it continues to perform its basic functions; however, it will not be able to provide a good user experience. Therefore, non-functional requirements are as important as functional requirements for successful product adoption.

- **Accuracy:** Power measurement error within 1% (PZEM-004T spec); forecast MAPE below 15% for daily usage.

- **Safety:** Sensors installed in enclosures by a licensed electrician; isolation from mains.
- **Reliability:** Local buffering during network loss.
- **Privacy:** Household data visible only to the household account.
- **Usability:** Simple charts and plain-language tips.
- **Performance:** Dashboard updates every 10 seconds.

### 3.3 User Requirements

- See current power use and today's units
- Know the expected bill for this cycle
- Get alerts for unusual consumption
- Know which appliance uses most energy
- Get simple tips to save money

### 3.4 System Requirements

- ESP32 + PZEM-004T whole-home node; optional smart plugs
- MQTT broker
- Python Flask server with Prophet and scikit-learn
- Time-series storage (SQLite/InfluxDB)
- Chart.js web dashboard

## CHAPTER IV

### SOFTWARE REQUIREMENT SPECIFICATION

#### 4.1 Hardware Specification

|                       |                                            |
|-----------------------|--------------------------------------------|
| System                | : Intel i3 Processor 2.4 GHz or above      |
| Hard Disk             | : 256 GB                                   |
| RAM                   | : 8 GB                                     |
| Monitor               | : 15" colour monitor                       |
| Energy node           | : ESP32, PZEM-004T v3.0 with CT, enclosure |
| Smart plug (optional) | : ESP32-C3, ACS712-20A, relay, 16A socket  |

#### 4.2 IoT Hardware Components

| S.No | Component                   | Purpose                                        |
|------|-----------------------------|------------------------------------------------|
| 1    | ESP32 DevKit                | Reads sensors, publishes MQTT                  |
| 2    | PZEM-004T v3.0 with 100A CT | Whole-home voltage, current, power, energy, PF |
| 3    | ACS712-20A                  | Appliance current in smart plug                |
| 4    | 5V relay module             | Optional remote switching of smart plug        |
| 5    | HLK-PM01 AC-DC module       | 5V supply from mains (enclosed)                |

#### 4.3 Software Specification

|                  |                                            |
|------------------|--------------------------------------------|
| Operating System | : Windows 10 / Ubuntu 22.04                |
| Firmware         | : Arduino IDE (C++), PZEM004Tv30 library   |
| Protocol         | : MQTT                                     |
| Back End         | : Python 3.10, Flask                       |
| ML               | : Prophet, scikit-learn, TensorFlow (LSTM) |
| Database         | : SQLite / InfluxDB                        |
| Front End        | : HTML, Chart.js                           |

## **CHAPTER V**

### **SYSTEM ANALYSIS**

#### **5.1 Feasibility Study**

A feasibility study is carried out to find whether the proposed WattWise system is worth developing. Three key considerations are involved in the feasibility analysis: technical, economic and operational feasibility.

##### **TECHNICAL FEASIBILITY**

PZEM-004T modules provide accurate readings over a simple serial interface; ESP32 and MQTT are reliable. Prophet and Isolation Forest are easy to use. The project is technically feasible.

##### **ECONOMIC FEASIBILITY**

The whole-home node costs about Rs. 1,500 and a smart plug about Rs. 700. Savings of even a few percent on monthly bills recover the cost within months. It is economically feasible.

##### **OPERATIONAL FEASIBILITY**

Once installed by an electrician, the system needs no user effort; the dashboard and alerts are easy to understand. It is operationally feasible.

#### **5.2 About the Software**

##### **ESP32 MICROCONTROLLER**

The ESP32 is a low-cost, low-power system-on-chip microcontroller from Espressif Systems with integrated Wi-Fi and dual-mode Bluetooth. It has a dual-core Tensilica Xtensa LX6 processor running up to 240 MHz, 520 KB SRAM, and many GPIO, ADC, I2C, SPI and UART pins.

The ESP32 can be programmed using the Arduino IDE in C/C++ or with MicroPython. Its built-in Wi-Fi lets sensor readings be sent directly to a cloud server using

MQTT or HTTP without any additional module.

- Built-in Wi-Fi and Bluetooth
- 12-bit ADC for analog sensors
- Deep-sleep mode for battery operation
- Costs less than Rs. 500

### **ENERGY SENSORS (PZEM-004T / ACS712)**

The PZEM-004T v3.0 is an AC communication module that measures voltage, current, active power, energy, frequency and power factor, and sends readings over a UART (Modbus-RTU) interface. The ACS712 is a Hall-effect based current sensor for measuring the current of individual appliances.

These sensors are connected to an ESP32 which publishes readings every few seconds to the cloud.

### **MQTT PROTOCOL**

MQTT (Message Queuing Telemetry Transport) is a lightweight publish-subscribe messaging protocol designed for constrained devices and low-bandwidth, high-latency or unreliable networks. It is an OASIS standard and is widely used in IoT.

Devices publish messages to topics on a broker (such as Mosquitto or HiveMQ), and any application subscribed to that topic receives the message instantly. MQTT supports three Quality of Service levels and retained messages.

- Very small packet overhead
- Publish-subscribe decoupling
- Works well on poor mobile networks

### **PYTHON**

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. Its design philosophy emphasises code

readability through significant indentation, and its simple syntax allows developers to express concepts in fewer lines of code than languages such as C++ or Java.

Python supports multiple programming paradigms including procedural, object-oriented and functional programming. It has a very large standard library and an ecosystem of third-party packages available through the Python Package Index (PyPI), which makes it the most popular language for data science, machine learning, automation and back-end web development.

- Easy to learn, read and maintain
- Huge library support for AI, ML and data processing (NumPy, Pandas, scikit-learn)
- Platform independent - runs on Windows, Linux and macOS
- Strong community support and documentation

## **PROPHET / ARIMA**

Prophet is an open-source forecasting procedure developed by Meta. It fits an additive model in which non-linear trends are combined with yearly, weekly and daily seasonality plus holiday effects. It works well with time series that have strong seasonal effects and several seasons of historical data.

ARIMA (AutoRegressive Integrated Moving Average) is a classical statistical model available in the statsmodels library and is used as a baseline for comparison.

## **SCIKIT-LEARN**

Scikit-learn is a free machine learning library for Python built on NumPy, SciPy and Matplotlib. It provides simple and efficient tools for predictive data analysis including classification, regression, clustering and dimensionality reduction.

It contains implementations of popular algorithms such as Random Forest, Support Vector Machine, Logistic Regression, k-Nearest Neighbours and Isolation Forest, along with utilities for train-test splitting, cross-validation, feature scaling, pipelines and evaluation metrics such as accuracy, precision, recall and F1-score.

- Consistent fit / predict API for all models
- Built-in preprocessing and model selection tools
- Well documented and production tested

## **FLASK**

Flask is a lightweight WSGI web application framework written in Python. It is classified as a micro-framework because it does not require particular tools or libraries; it has no database abstraction layer or form validation built in, but extensions can add these features as needed.

Flask provides URL routing, request handling, Jinja2 templating and a development server. Because it is small and simple, it is ideal for exposing a trained machine learning model as a REST API or building a small web application quickly.

- Minimal and easy to learn
- Jinja2 templates for HTML pages
- Large set of extensions (Flask-Login, Flask-SQLAlchemy, Flask-CORS)

## **CHART.JS**

Chart.js is a free, open-source JavaScript library for data visualisation that supports eight chart types including bar, line, pie and radar charts. Charts are rendered on an HTML5 canvas and are responsive by default.

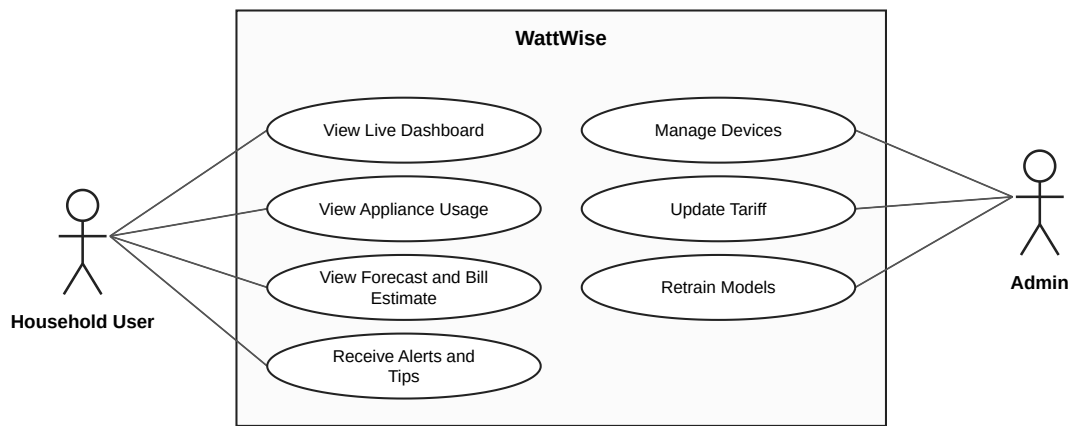
## CHAPTER VI

### SYSTEM DESIGN

System design is the process of defining the architecture, components, modules, interfaces and data of a system to satisfy the specified requirements. The design of WattWise is presented using UML use case diagram, system architecture, data flow diagrams, ER diagram and table design.

#### 6.1 UML Diagram

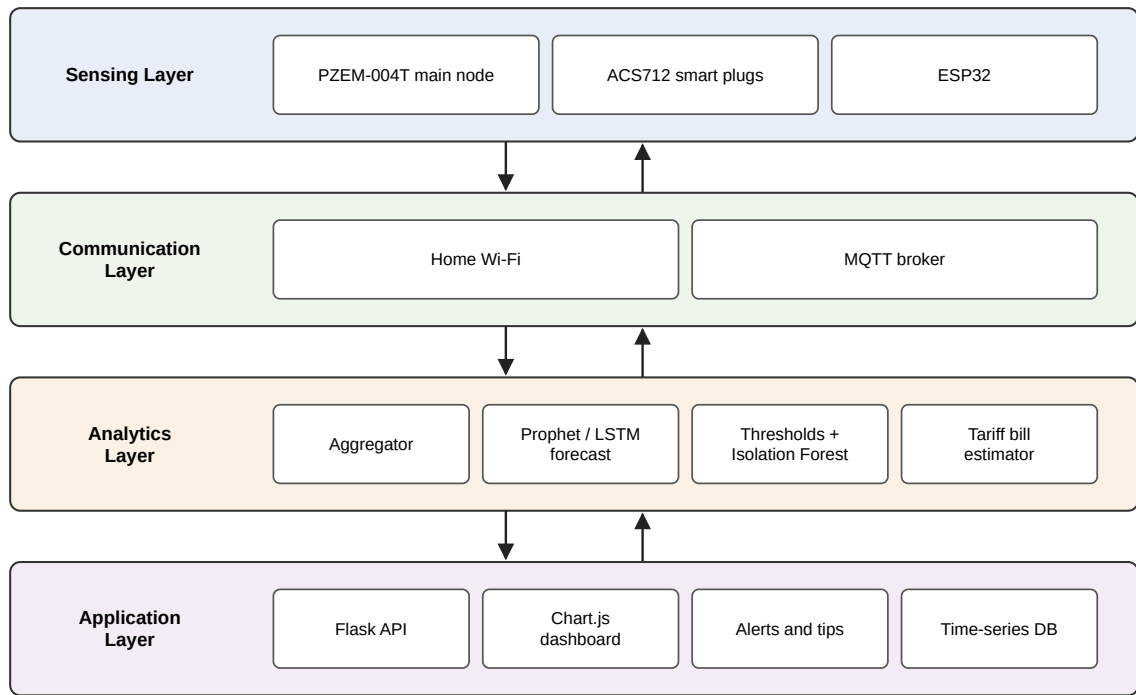
##### USE CASE DIAGRAM



**Fig 1: Use case diagram of WattWise**

#### 6.2 System Architecture

WattWise follows an IoT architecture: energy sensing nodes publish readings over MQTT; a Python server aggregates data and runs forecasting and anomaly models; a web dashboard presents results and alerts.

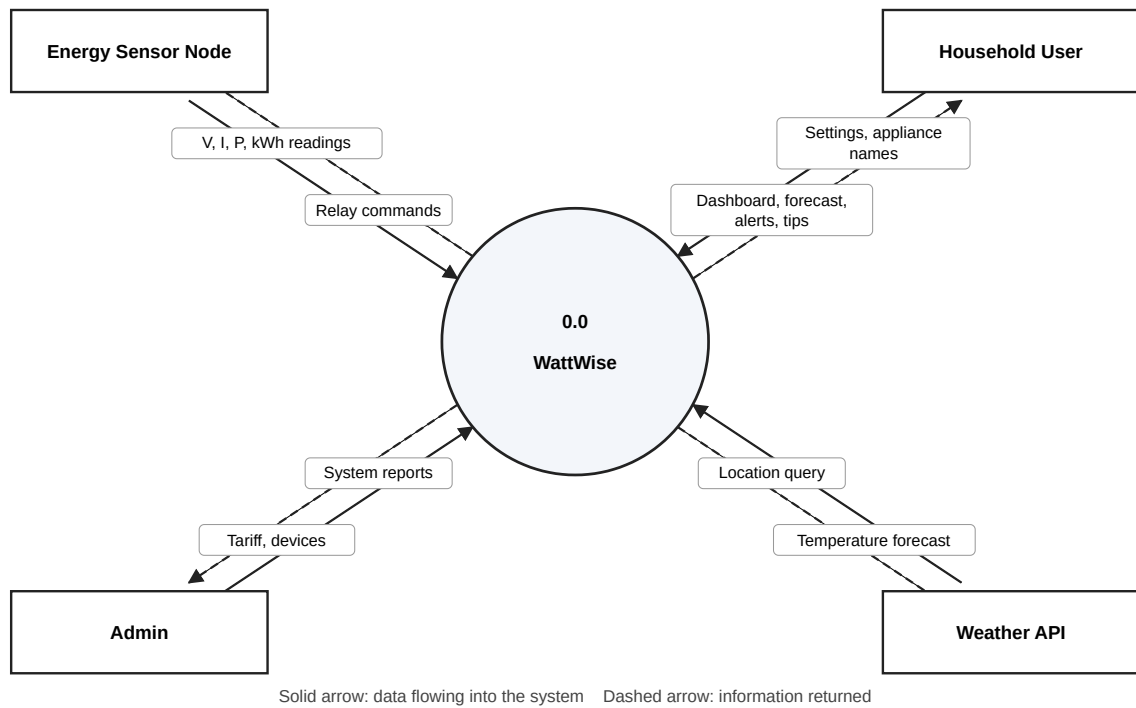


**Fig 2: System architecture**

### 6.3 Data Flow Diagram

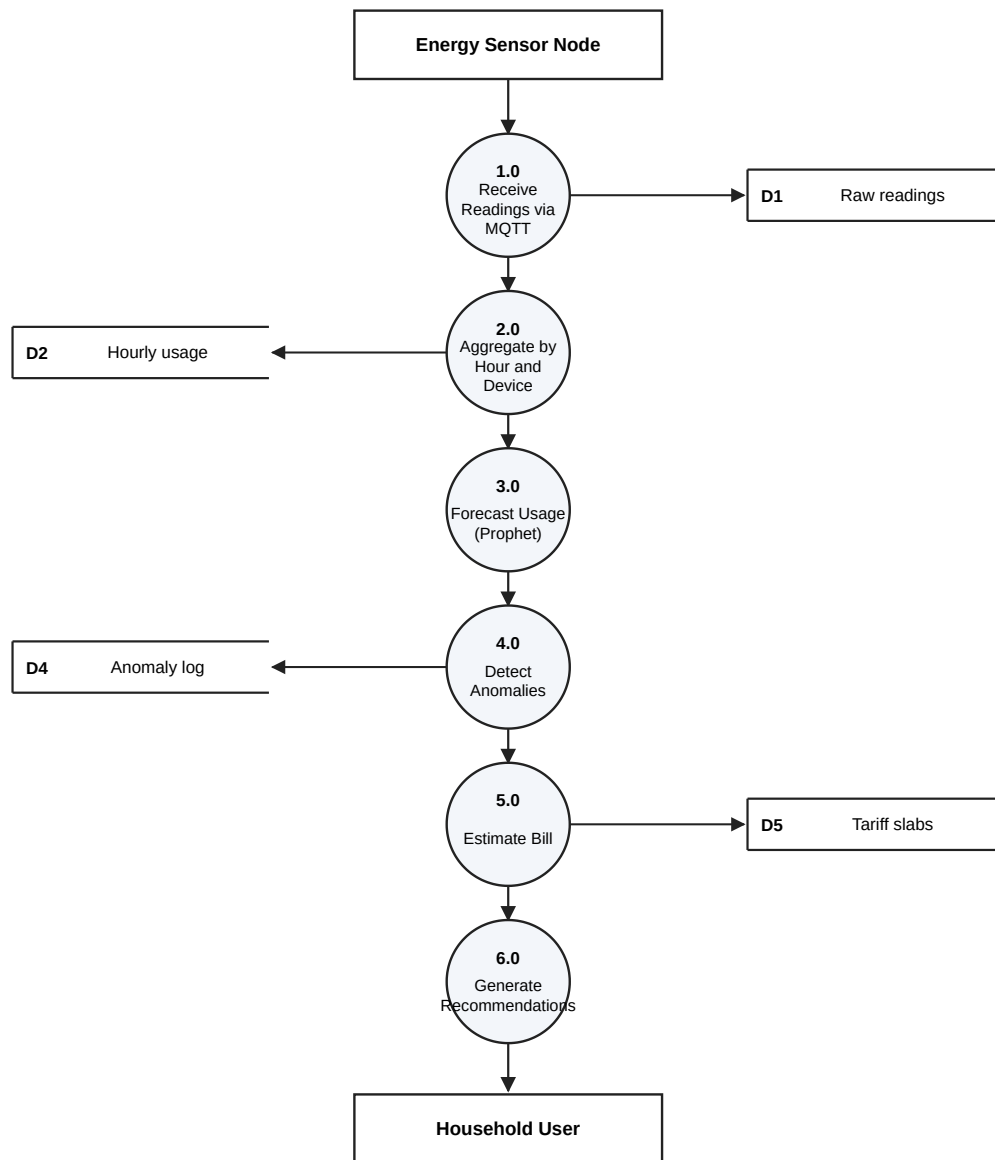
A Data Flow Diagram (DFD) is a graphical representation of the flow of data through an information system. The Level 0 DFD (context diagram) shows the whole system as a single process with its external entities, and the Level 1 DFD breaks it into its major processes and data stores.

## LEVEL 0 DFD (CONTEXT DIAGRAM)



**Fig 3: Level 0 data flow diagram**

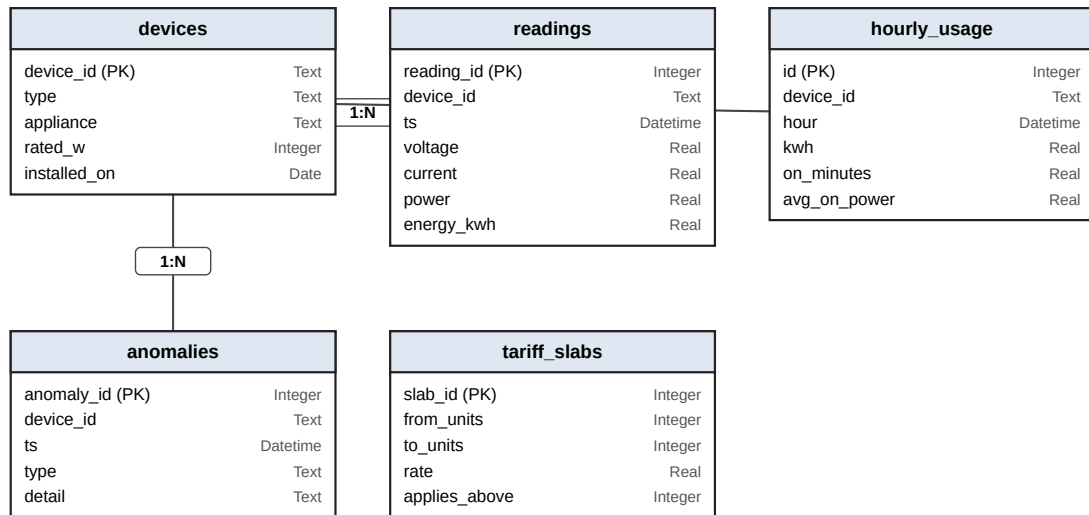
## LEVEL 1 DFD



**Fig 4: Level 1 data flow diagram**

## 6.4 ER Diagram

The Entity Relationship (ER) diagram shows the entities stored in the database, their key attributes and the relationships between them. 1:N indicates a one-to-many relationship.



**Fig 5: Entity relationship diagram**

## 6.5 Table Design

### DEVICES

| FIELD NAME   | DATA TYPE | DESCRIPTION       |
|--------------|-----------|-------------------|
| device_id    | Text      | Primary key       |
| type         | Text      | main / plug       |
| appliance    | Text      | e.g. Refrigerator |
| rated_w      | Integer   | Rated power       |
| installed_on | Date      | Installation date |

### READINGS

| FIELD NAME | DATA TYPE | DESCRIPTION            |
|------------|-----------|------------------------|
| reading_id | Integer   | Primary key            |
| device_id  | Text      | Foreign key to devices |
| ts         | Datetime  | Time                   |
| voltage    | Real      | Volts                  |
| current    | Real      | Amps                   |
| power      | Real      | Watts                  |
| energy_kwh | Real      | Cumulative kWh         |

## HOURLY\_USAGE

| FIELD NAME   | DATA TYPE | DESCRIPTION            |
|--------------|-----------|------------------------|
| id           | Integer   | Primary key            |
| device_id    | Text      | Foreign key to devices |
| hour         | Datetime  | Hour                   |
| kwh          | Real      | Energy in the hour     |
| on_minutes   | Real      | Minutes switched on    |
| avg_on_power | Real      | Average power when on  |

## ANOMALIES

| FIELD NAME | DATA TYPE | DESCRIPTION                         |
|------------|-----------|-------------------------------------|
| anomaly_id | Integer   | Primary key                         |
| device_id  | Text      | Foreign key to devices              |
| ts         | Datetime  | Time                                |
| type       | Text      | spike / sustained / appliance_fault |
| detail     | Text      | Description                         |

## TARIFF\_SLABS

| FIELD NAME    | DATA TYPE | DESCRIPTION                                |
|---------------|-----------|--------------------------------------------|
| slab_id       | Integer   | Primary key                                |
| from_units    | Integer   | From (units per cycle)                     |
| to_units      | Integer   | To                                         |
| rate          | Real      | Rs per unit                                |
| applies_above | Integer   | Total units above which slab table applies |

## 6.6 Algorithm Design

The analytics pipeline runs continuously for anomalies and once every hour for forecasting and bill estimation.

1. **Measure power with current/voltage sensor** – PZEM-004T gives V, I, P, kWh and PF over Modbus-RTU; ACS712 gives RMS current for plugs ( $P = V \times I \times \text{assumed PF}$ ).
2. **Send readings to cloud at intervals** – Published every 10 seconds as JSON over MQTT.
3. **Aggregate by hour and device** – kWh per hour from cumulative energy differences; on-minutes and average on-power for appliances.
4. **Forecast usage with LSTM/Prophet** – Prophet with daily and weekly seasonality and temperature regressor forecasts the next 24 hours and the remaining days of the billing cycle.
5. **Detect anomalies using statistical thresholds and Isolation Forest** – Hourly kWh above mean + 3 sigma for that hour-of-week is a spike; Isolation Forest on appliance features (duty cycle, on-power, cycles per hour) detects faults.
6. **Estimate bill using tariff slabs** – Predicted cycle units are applied to the domestic slab table to compute the expected bill.
7. **Recommend load shifting and appliance tips** – Rules based on usage patterns, e.g., heavy loads during peak hours, high standby power, rising refrigerator duty cycle, slab-crossing warning.
8. **Show daily and monthly dashboard** – Chart.js charts updated through a REST API.

## PSEUDO CODE

```

EVERY HOUR:
  hourly <- AGGREGATE(readings, by device, hour)
  FOR each device d:
    mu, sigma <- STATS(hourly[d], same hour-of-week, last 8 weeks)
    IF hourly[d].kwh > mu + 3*sigma THEN ANOMALY(d, "spike")
    IF d.type = plug AND IFOREST[d].predict([duty, on_power, cycles]) = -1 THEN
      ANOMALY(d, "appliance_fault")
    fc <- PROPHET.predict(next 24 h + rest of cycle, temperature forecast)
    units <- used_so_far + SUM(fc)
    bill <- SLAB_BILL(units)
    IF units crosses next slab boundary THEN TIP("Reduce X units to stay in lower slab")
  TIPS <- RULES(peak_usage, standby, appliance trends)

```

## **6.7 Modules**

The modules are divided between the Household User and the Admin.

### **MODULES**

#### **HOUSEHOLD USER**

1. VIEW LIVE DASHBOARD
2. VIEW APPLIANCE USAGE
3. VIEW FORECAST AND BILL ESTIMATE
4. RECEIVE ALERTS AND TIPS

#### **ADMIN**

1. MANAGE DEVICES
2. UPDATE TARIFF
3. RETRAIN MODELS

### **HOUSEHOLD USER**

#### **VIEW LIVE DASHBOARD**

Shows current power (W), today's units (kWh), this billing cycle's units and the estimated bill.

#### **VIEW APPLIANCE USAGE**

Shows consumption by appliance (for smart plug devices) and the share of each in the total.

#### **VIEW FORECAST AND BILL ESTIMATE**

Shows tomorrow's predicted hourly usage and the predicted cycle total with slab information.

#### **RECEIVE ALERTS AND TIPS**

Receives alerts for abnormal usage and suggestions to reduce cost.

## **ADMIN**

### **MANAGE DEVICES**

Registers energy nodes and smart plugs and assigns appliance names.

### **UPDATE TARIFF**

Updates tariff slabs and fixed charges as per the electricity board.

### **RETRAIN MODELS**

Retrains forecasting and anomaly models periodically.

## **CHAPTER VII**

### **IMPLEMENTATION AND TESTING**

#### **7.1 Introduction**

The WattWise energy node firmware was written for the ESP32 using the PZEM004Tv30 Arduino library. A Python Flask application subscribes to MQTT, stores readings in SQLite (or InfluxDB), aggregates hourly data and runs Prophet forecasting and Isolation Forest anomaly detection using APScheduler. The dashboard uses Chart.js.

#### **7.2 Implementation**

The models were developed using the public UK-DALE dataset for appliance behaviour and three months of data collected from two volunteer households in Tamil Nadu. A refrigerator fault was simulated by leaving the door slightly open, and a water heater thermostat fault by bypassing the thermostat for a short supervised test.

Testing is a process of running software with the intention of finding errors. The software which has been developed has to be tested to prove its validity. Testing is considered to be the least creative phase of the whole cycle of system design; in the real sense it is the phase which helps to bring out the creativity of the other phases. Testing a program is the first step in the debugging process.

A good testing suite is vital because all possible execution paths inside the code must be tested in order to have any hope that it is bug free. Testing of the system is done to ensure the integrity of the system. The aim of testing is to prove that the developed system addresses the predefined processing requirements and will perform reliably and efficiently during operation.

#### **PURPOSES OF TESTING**

- To affirm the quality of the project.
- To find and eliminate any errors from previous stages.
- To validate the software against its requirements.

- To ensure the reliability of the system.

## **UNIT TESTING**

In the lines of this strategy all the individual functions and modules were put to the test independently. By following this strategy all the errors in coding were identified and corrected. This method was applied in combination with white box and black box testing techniques to find the errors in each module.

## **INTEGRATION TESTING**

This testing strategy has two different approaches, namely the top-down approach, in which integration is carried out from the top-level module to the bottom, and the bottom-up approach, in which integration is carried out from the low-level modules to the top. The modules were tested using the bottom-up approach by introducing stubs for the top-level functions. This test was used to identify errors in the interfaces and in passing parameters between functions.

## **VALIDATION TESTING**

The main aim of this testing is to verify that the software system does what it was designed for. Alpha testing and beta testing were conducted to ensure the validity of the system. Alpha testing was done at the developer's site and beta testing was carried out by sample end users.

## **SYSTEM TESTING**

The idea of applying this strategy was to put the system through a series of different tests whose primary purpose is to fully exercise the computer-based system. Although each test has a different purpose, all work to verify that all system elements have been properly integrated and perform their allocated functions. Security testing and performance testing were also conducted.

## **ACCEPTANCE TESTING**

Acceptance testing involves planning and execution of functional tests, performance tests and stress tests to verify that the implemented system satisfies its

requirements. Typically, the acceptance test incorporates test cases developed during unit testing and integration testing.

### 7.3 Test Cases

| Test ID | Test Description   | Input                       | Expected Output                 | Result |
|---------|--------------------|-----------------------------|---------------------------------|--------|
| TC01    | Power reading      | 1000 W heater               | Reading 990 - 1010 W            | Pass   |
| TC02    | MQTT upload        | Node running                | Dashboard updates every 10 s    | Pass   |
| TC03    | Hourly aggregation | Known energy test           | kWh matches meter within 1%     | Pass   |
| TC04    | Forecast           | 60 days history             | Next-day forecast with interval | Pass   |
| TC05    | Spike anomaly      | Heater left on 3 h at night | Spike alert                     | Pass   |
| TC06    | Appliance fault    | Fridge door ajar            | Duty cycle anomaly alert        | Pass   |
| TC07    | Bill estimate      | Predicted 520 units         | Bill as per slab table          | Pass   |
| TC08    | Slab warning       | Forecast crosses 500 units  | Tip to reduce usage             | Pass   |

## PERFORMANCE EVALUATION

Forecasting accuracy (daily kWh, 30-day test period):

| Model                            | MAE (kWh/day) | MAPE  |
|----------------------------------|---------------|-------|
| Same day last week               | 1.92          | 21.4% |
| ARIMA                            | 1.48          | 16.5% |
| Prophet + temperature (proposed) | 1.07          | 11.8% |
| LSTM                             | 1.12          | 12.6% |

### 7.4 Source Code

#### energy\_node.ino (ESP32 + PZEM-004T)

```
#include <PZEM004Tv30.h>
#include <WiFi.h>
#include <PubSubClient.h>
```

```

PZEM004Tv30 pzem(Serial2, 16, 17);    // RX, TX
WiFiClient net; PubSubClient mqtt(net);
const char* NODE = "home-main";

void setup() {
  WiFi.begin("HOME_WIFI", "password");
  while (WiFi.status() != WL_CONNECTED) delay(300);
  mqtt.setServer("192.168.1.10", 1883);
}

void loop() {
  if (!mqtt.connected()) mqtt.connect(NODE, "wattwise", "secret");
  float v = pzem.voltage(), i = pzem.current(), p = pzem.power(), e =
pzem.energy(), pf = pzem.pf();
  if (!isnan(v)) {
    char msg[160];
    snprintf(msg, sizeof msg, "
{"v\\":%.1f,\\\"i\\":%.3f,\\\"p\\":%.1f,\\\"kwh\\":%.3f,\\\"pf\\":%.2f}", v, i, p, e, pf);
    mqtt.publish("wattwise/home-main", msg);
  }
  mqtt.loop();
  delay(10000);
}

```

## analytics.py

```

import sqlite3, pandas as pd, numpy as np, joblib
from prophet import Prophet

con = sqlite3.connect("wattwise.db")
SLABS_LE_500 = [(0, 100, 0.0), (101, 200, 2.35), (201, 400, 4.70), (401, 500,
6.30)]
SLABS_GT_500 = [(0, 100, 0.0), (101, 400, 4.70), (401, 500, 6.30), (501, 600,
8.40),
                (601, 800, 9.45), (801, 1000, 10.50), (1001, 10**6, 11.55)] #
example bi-monthly slabs (update from TNEB tariff order)

def hourly(device="home-main"):
    df = pd.read_sql("SELECT ts, energy_kwh FROM readings WHERE device_id=?
ORDER BY ts", con,
                    params=(device,), parse_dates=["ts"]).set_index("ts")
    return df.energy_kwh.resample("1h").last().diff().clip(lower=0).dropna()

def forecast(series, temps, horizon_h):
    df = series.reset_index(); df.columns = ["ds", "y"]
    df = df.merge(temps, on="ds", how="left").ffill()
    m = Prophet(daily_seasonality=True, weekly_seasonality=True,
yearly_seasonality=False)
    m.add_regressor("temp"); m.fit(df)
    fut = m.make_future_dataframe(periods=horizon_h, freq="h").merge(temps,
on="ds", how="left").ffill()
    return m.predict(fut).tail(horizon_h)[["ds", "yhat", "yhat_lower",
"yhat_upper"]]

def bill(units):
    slabs = SLABS_LE_500 if units <= 500 else SLABS_GT_500
    total = 0.0
    for lo, hi, rate in slabs:
        if units >= lo:
            total += (min(units, hi) - max(lo, 1) + 1) * rate if lo > 0 else 0

```

```

    return round(total, 2)

def spikes(series):
    df = series.to_frame("kwh"); df["how"] = df.index.dayofweek * 24 +
df.index.hour
    stats = df.iloc[:-24].groupby("how").kwh.agg(["mean", "std"])
    last = df.iloc[-24:].join(stats, on="how")
    return last[last.kwh > last["mean"] + 3 * last["std"].fillna(0.1)]

def appliance_fault(device):
    feats = pd.read_sql("""SELECT on_minutes/60.0 AS duty, avg_on_power, kwh
FROM hourly_usage
                        WHERE device_id=? ORDER BY hour DESC LIMIT 24""",
con, params=(device,))
    model = joblib.load(f"models/iforest_{device}.pkl")
    return (model.predict(feats) == -1).mean() > 0.5      # majority of last day
abnormal

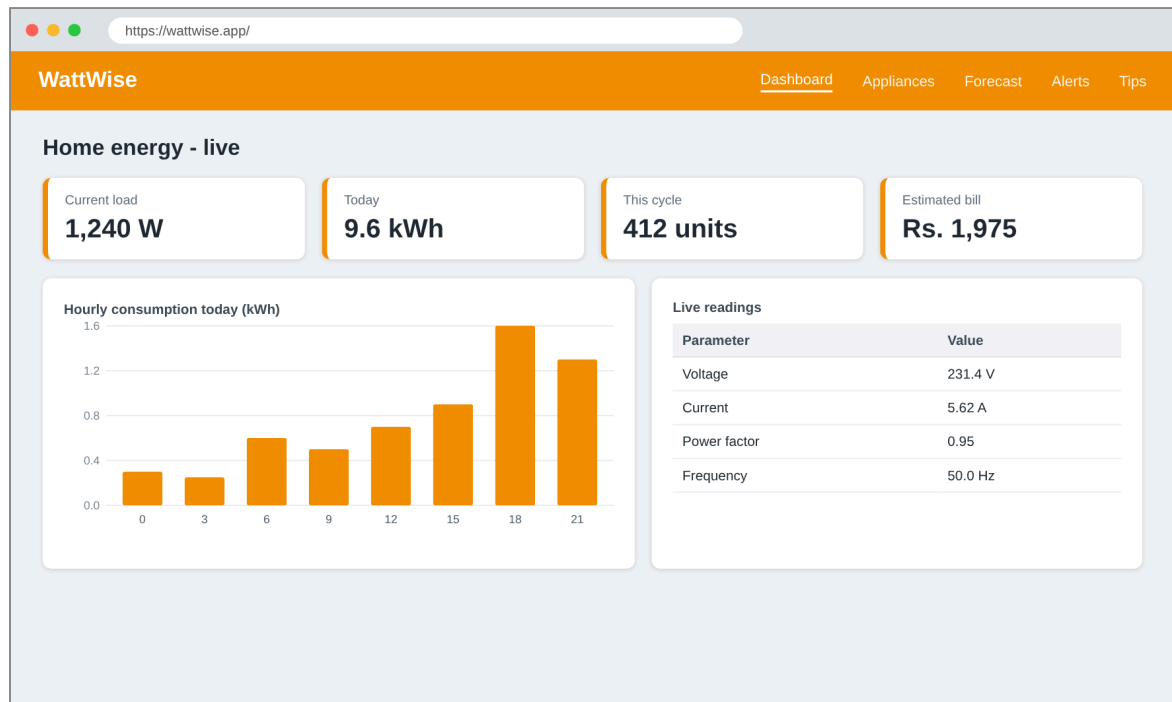
def tips(series, predicted_units):
    out = []
    peak = series[series.index.hour.isin(range(18, 22))].sum() /
max(series.sum(), 1e-6)
    if peak > 0.35: out.append("35%+ of your use is between 6-10 PM. Run washing
machine and pump before 5 PM.")
    night_base = series[series.index.hour.isin([2, 3, 4])].mean()
    if night_base > 0.15: out.append(f"Standby load at night is
{night_base*1000:.0f} W. Switch off TV/set-top box at the plug.")
    if 480 <= predicted_units <= 540:
        out.append(f"You are predicted to use {predicted_units:.0f} units.
Staying under 500 units keeps you in the lower tariff table.")
    return out

```

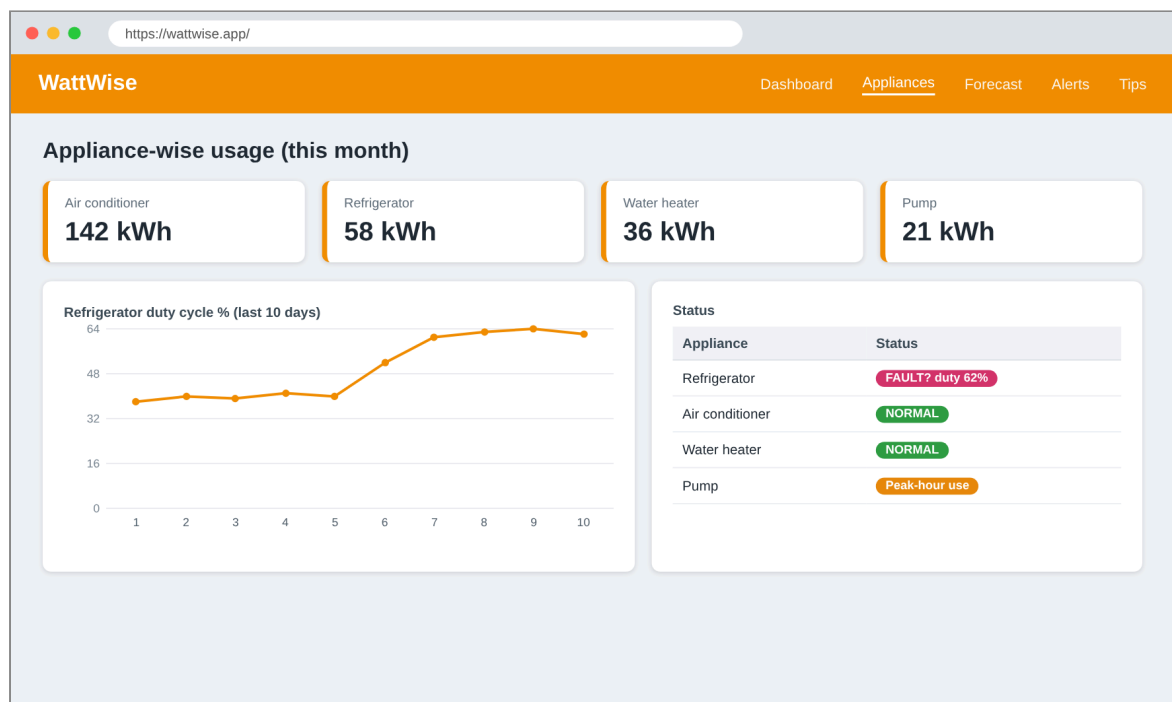
## 7.5 Screen Layout

### SCREENSHOTS:

## ENERGY DASHBOARD



## APPLIANCE BREAKDOWN



## FORECAST AND BILL ESTIMATE

WattWise

DashboardAppliancesForecastAlertsTips

Forecast for this billing cycle

Next 24 hours

Predicted: 10.4 kWh

Range: 9.1 - 11.8 kWh

Peak hour: 7 PM (1.7 kWh)

Tomorrow max temp: 36 C

Predicted 528 units - crosses 500-unit limit

|                         |                                |
|-------------------------|--------------------------------|
| Used so far (45 days)   | 412 units                      |
| Remaining days          | 15                             |
| Predicted total         | 528 units                      |
| Estimated bill          | Rs. 2,870                      |
| If kept under 500 units | Rs. 1,980 (save about Rs. 890) |

Reduce about 2 units/day: set AC to 26 C and use timer.

Run the pump and washing machine before 5 PM.

## ALERTS AND TIPS

WattWise

DashboardAppliancesForecastAlertsTips

Alerts and saving tips

| Time        | Type      | Message                                                                   | Status |
|-------------|-----------|---------------------------------------------------------------------------|--------|
| 14-10 07:00 | APPLIANCE | Refrigerator running 62% of time (normal 40%). Check door seal / defrost. | New    |
| 13-10 23:10 | SPIKE     | Water heater ON for 3 h at night (unusual).                               | Seen   |
| 12-10 20:00 | TIP       | 38% of usage is 6-10 PM. Shift pump to afternoon.                         | Seen   |
| 11-10 09:00 | TIP       | Standby load 120 W at night. Switch off devices at plug.                  | Done   |

## **CHAPTER VIII**

### **CONCLUSION AND FUTURE ENHANCEMENT**

#### **8.1 Conclusion**

The Smart Household Energy Forecasting and Anomaly Detection system gives households real-time and appliance-level insight into their electricity consumption. The Prophet model with temperature as a regressor forecast daily consumption with a MAPE of 11.8%, enabling bill prediction and timely warnings before crossing tariff slabs. Anomaly detection identified simulated appliance faults such as a refrigerator with its door ajar.

Personalised tips on load shifting and standby power help users reduce their bills, and early fault detection prevents waste and appliance damage. The low-cost IoT design makes the system practical for homes and small shops.

#### **8.2 Future Enhancement**

- Non-intrusive load monitoring (NILM) to identify appliances from the main meter alone.
- Integration with smart meters of the electricity board.
- Automatic control of smart plugs for peak-hour load shifting.
- Rooftop solar generation forecasting and net-metering optimisation.
- EV charging scheduling at off-peak hours.

## **CHAPTER IX**

### **BIBLIOGRAPHY**

1. Hart, G. W., "Nonintrusive appliance load monitoring", Proceedings of the IEEE, Vol. 80, No. 12, pp. 1870-1891, 1992.
2. Kelly, J., and Knottenbelt, W., "The UK-DALE dataset, domestic appliance-level electricity demand and whole-house demand from five UK homes", Scientific Data, Vol. 2, 150007, 2015.
3. Kong, W., Dong, Z. Y., Jia, Y., Hill, D. J., Xu, Y., and Zhang, Y., "Short-Term Residential Load Forecasting Based on LSTM Recurrent Neural Network", IEEE Transactions on Smart Grid, Vol. 10, No. 1, pp. 841-851, 2019.
4. Taylor, S. J., and Letham, B., "Forecasting at Scale", The American Statistician, Vol. 72, No. 1, pp. 37-45, 2018.
5. Liu, F. T., Ting, K. M., and Zhou, Z.-H., "Isolation Forest", Proc. IEEE ICDM, 2008.
6. Darby, S., "The Effectiveness of Feedback on Energy Consumption: A Review for DEFRA of the Literature on Metering, Billing and Direct Displays", Environmental Change Institute, University of Oxford, 2006.

#### **WEBSITE REFERENCE**

1. <https://www.tnebltd.gov.in> (TANGEDCO tariff)
2. <https://facebook.github.io/prophet>
3. <https://github.com/mandulaj/PZEM-004T-v30>
4. <https://www.chartjs.org>